

The Fundamentals of Programming

Creating Functions



Objectives

What you will learn:

1. How to create your own function.
2. How to put functions together.
3. How to get information back from a function using return.

Functions

Function arguments

We have seen how functions can have one or more values. These values we **pass** into a function are called **arguments**.

Notice how some functions take only one argument, while others can take any number of them, like min and max.

```
print("hi")  
abs(-150)  
int(6.6)  
float(1)  
min(1, 2, 3, 4)  
max(1, 2, 3, 4)
```

Functions

Function return values

Functions in Python will always return a value.

int takes an argument that is a number and returns an integer.

max takes many numbers and returns the largest.

What does **print** return?

```
print('hi')  
abs(-150)  
int(6.6)  
float(1)  
min(1, 2, 3, 4)  
max(1, 2, 3, 4)
```

```
print(print('hi'))
```

Making a function

Using the built-in functions is great and all, but the real power in programming comes from building your own functions.

Let's look at creating a simple function that prints a greeting.

```
def greeting():  
    print("Hello Mika")
```

def is a keyword (a special word that is part of Python) which tells Python you're starting a function.

() creates space for our arguments. Right now there are none.

: lets Python know you're done defining your function.

Indenting tells Python you're inside the function. When you unindent you're no longer in the function.

Making a function

Here we have three versions of the same program. Let's run through each and see how they differ.

```
def greeting():  
    print('Hello Mika')
```

What happens if you run this?

```
def greeting():  
    print('Hello Mika')  
greeting()
```

We call the function.

```
def greeting(name):  
    print('Hello ' + name)  
greeting('Mika')
```

What do you think this does?

Function definition

The setup

A function definition is what tells us how it can be used. It requires the **def** keyword, the function **name**, **parenthesis** **()**, 0 or more **parameters** and a **colon**:

```
def foo(a, b):
```

```
foo(5, 3)  
foo('hi', 'bye')  
foo(7.5, 3)
```

Notice how you can pass in many different types of variables and the function doesn't care.

Practice

Putting it together

In each of these, what part is in the function and what part is outside? What is printed to the screen in these programs?

```
def foo(a, b):  
    a = a + b  
    print(a * b)  
foo(1, 2)  
foo(2, 3)
```

```
def area(b, h):  
    print((b*h)/2)  
area(10, 3)  
area(3, 8)
```

```
def go(a, b, c):  
    a = b * a  
    print(a * c)  
go(3, 2, 'Hi')
```

Well, this is new

Returning values

An improvement

The first program is what we just saw, but there is a better way to write it using the **return** keyword.

```
def area(b, h):  
    print((b*h)/2)  
area(10, 3)
```

This works.
This works

```
def area(b, h):  
    return (b*h)/2  
print(area(10, 3))
```

But this is better.
Why?

Using functions together

Using return values instead of print statements means we can combine function any way we want and use the values how we want.

What will the output be from the program at the right?

```
def double(value):  
    return value * 2
```

```
def increase(value):  
    return value + 1
```

```
print(increase(double(5)))  
print(double(increase(6)))  
temp = double(10)  
print(double(temp))
```

Returning values

Digging in

To help you understand how this works, let's walk through how Python sees a print statement with multiple function calls.

```
def double(value):  
    return value * 2
```

```
def increase(value):  
    return value + 1
```

```
print(double(increase(5))) ← Call increase  
print(double(6))           ← Call double  
print(12)                  ← Call print
```

Practice

How many parameters do each of these functions have?

What is the first line of code that is run in this program? What is the first line read?

What is the output for each of the print statements?

```
def double(value):  
    return value * 2
```

```
def increase(value):  
    return value + 1
```

```
def both(value):  
    doubled = double(value + 3)  
    return increase(doubled)
```

```
print(double(4) + increase(2))  
print(double(increase(4)))  
print(both(10))  
print(both(both(2)))  
print(double(increase(both(5))))
```

Recap

We learned

1. How to define a function according to Python's rules.
2. The difference between argument and parameter
3. How to return values from a function.
4. How to use multiple functions together.